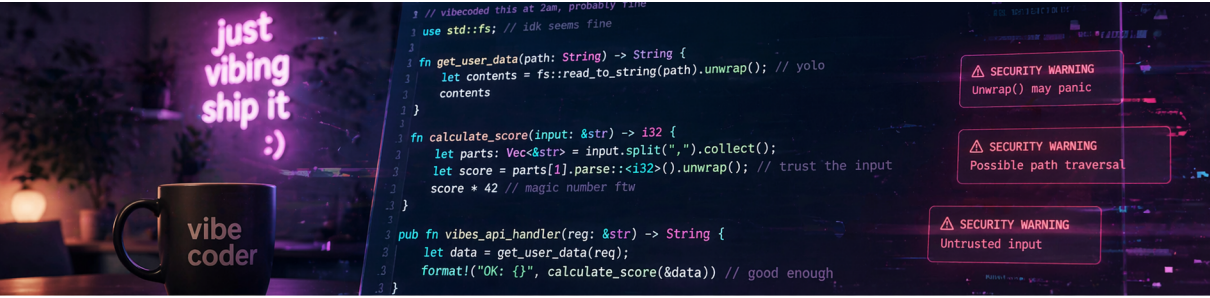
universität innsbruck

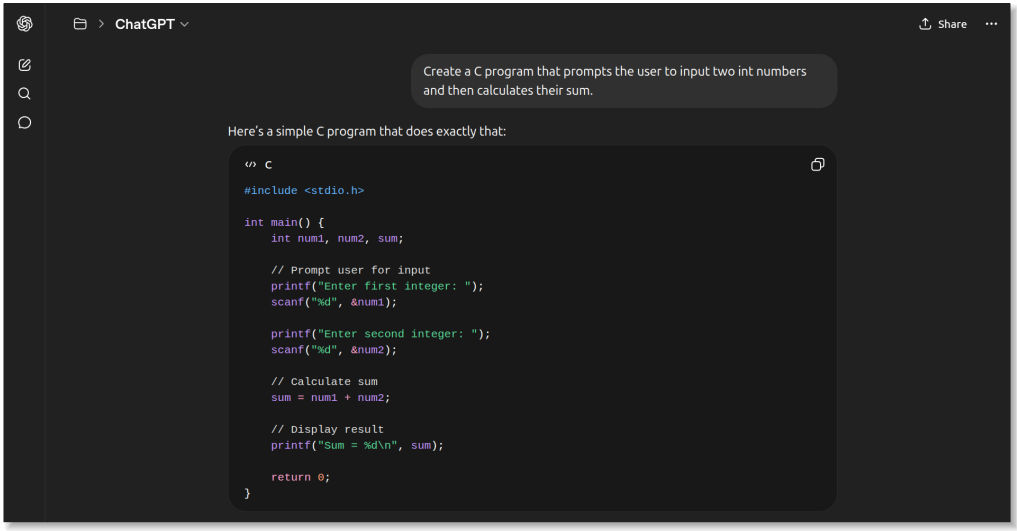


Vibe Coding and Software Vulnerabilities

Context Presentation: Vulnerabilities introduced by LLMs

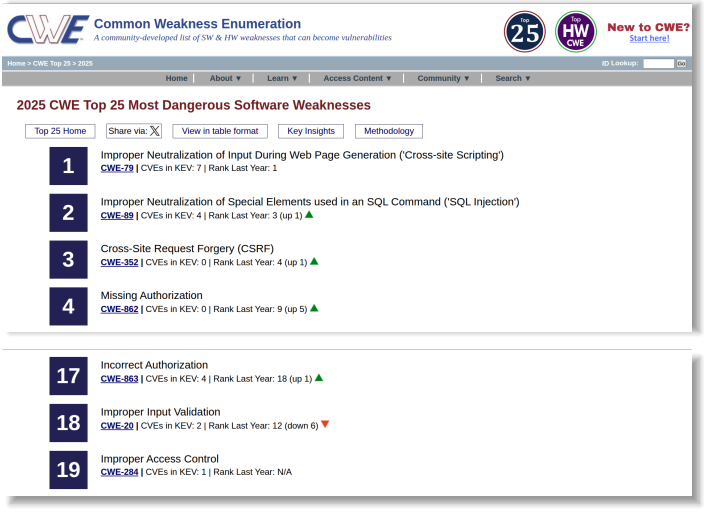
Stefan Wagner

Motivation



Idea from [1]. Generated by ChatGPT Instant 5.3 : `https://chatgpt.com/share/69eb9fca-a960-8388-a707-eb09f86575b7`

Motivation



MITRE: Common Weakness Enumeration (CWE) · `https://cwe.mitre.org/top25/archive/2025/2025_cwe_top25.html`

Vibe Coding & Software Vulnerabilities

Observation

- LLMs are now part of everyday programming
- Generated code can contain vulnerabilities
- Vulnerabilities may be missed during review
- Suggested fixes can be incomplete

Open question

Does this generalize across models, prompts, tasks, and programming languages?

Vibe Coding & Software Vulnerabilities

Observation

- LLMs are now part of everyday programming
- Generated code can contain vulnerabilities
- Vulnerabilities may be missed during review
- Suggested fixes can be incomplete

Open question

Does this generalize across models, prompts, tasks, and programming languages?

⇒ **Overview of current research and implications for Rust**

Systematic Literature Review (SLR)

From Vulnerabilities to Remediation

- Analysis of 102 studies on LLMs in code security
- Focus on “recent“ work, mainly 2024 – 2025

From Vulnerabilities to Remediation: A Systematic Literature Review of LLMs in Code Security

ENNA BASIC, Department of Computer Science, Örebro University, Sweden and Epiroc Rock Drills, Sweden
ALBERTO GIARETTA, Department of Computer Science, Örebro University, Sweden

Large Language Models (LLMs) have emerged as powerful tools for automating programming tasks, including security-related ones. However, they can also introduce vulnerabilities during code generation, fail to detect existing vulnerabilities, or report nonexistent ones. This systematic literature review investigates the security benefits and drawbacks of using LLMs for code-related tasks. In particular, it focuses on the types of vulnerabilities introduced by LLMs when generating code. Moreover, it analyzes the capabilities of LLMs to detect and fix vulnerabilities, and examines how prompting strategies impact these tasks. Finally, it examines how data poisoning attacks impact LLMs performance in the aforementioned tasks.

CCS Concepts: • **Security and privacy** → **Software security engineering**; **Vulnerability scanners**; • **Computing methodologies** → *Artificial intelligence*.

Additional Key Words and Phrases: Large Language Models, LLMs, Security Vulnerabilities, LLM-generated Code, Vulnerability Detection, Vulnerability Fixing, Prompt Engineering, Data Poisoning

Enna Basic and Alberto Giaretta. *ACM Comput. Surv.* 1, 1 (March 2026), 41 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Large Language Models (LLMs) are Machine Learning (ML) models designed for natural language processing [12]. Representative models such as ChatGPT [3], Llama [106] and GitHub Copilot [4] gained extreme popularity and recognition in the last couple of years due to their ability to perform well across different tasks. Throughout this paper, we use the term "LLM" to refer to large autoregressive Transformer language models that support prompt-driven, open-ended code generation and analysis. In contrast, we use Pre-trained Language Model (PLM) to refer to pre-trained (often smaller) encoder-only Transformer models (e.g., CodeBERT, GraphCodeBERT) that are typically fine-tuned for task-specific prediction or classification rather than open-ended generation. We treat these PLMs and other task-specific neural models as traditional ML baselines and include them only when they serve as comparison points for LLM-based approaches.

Among other capabilities, LLMs effectively translate natural language into code, assist in code comprehension, and answer questions related to code [43]. Many developers utilize these abilities to optimize coding processes, reducing development time and thereby improving productivity. According to GitHub statistics, GitHub Copilot is now responsible for generating approximately 46% of code and boosts developers' coding speed by up to 55% [23]. In addition to their coding skills, LLMs can be powerful tools for vulnerability detection and fixing, particularly when provided with detailed prompts and clear background information [32, 129].

Authors' Contact Information: Enna Basic, Department of Computer Science, Örebro University, Örebro, Sweden and Epiroc Rock Drills, Örebro, Sweden; Alberto Giaretta, Department of Computer Science, Örebro University, Örebro, Sweden.

ACM 1557-7341/2026/3-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Basic & Giaretta · Preprint 2026 [2]

Systematic Literature Review (SLR)

From Vulnerabilities to Remediation

- Analysis of 102 studies on LLMs in code security
- Focus on “recent“ work, mainly 2024 – 2025

SLR Scope

1. Vulnerabilities introduced by LLMs
2. LLM-based vulnerability detection and fixing
3. Prompting strategies
4. Data poisoning attacks

From Vulnerabilities to Remediation: A Systematic Literature Review of LLMs in Code Security

ENNA BASIC, Department of Computer Science, Örebro University, Sweden and Epiroc Rock Drills, Sweden
ALBERTO GIARETTA, Department of Computer Science, Örebro University, Sweden

Large Language Models (LLMs) have emerged as powerful tools for automating programming tasks, including security-related ones. However, they can also introduce vulnerabilities during code generation, fail to detect existing vulnerabilities, or report nonexistent ones. This systematic literature review investigates the security benefits and drawbacks of using LLMs for code-related tasks. In particular, it focuses on the types of vulnerabilities introduced by LLMs when generating code. Moreover, it analyzes the capabilities of LLMs to detect and fix vulnerabilities, and examines how prompting strategies impact these tasks. Finally, it examines how data poisoning attacks impact LLMs performance in the aforementioned tasks.

CCS Concepts: • **Security and privacy** → **Software security engineering**; **Vulnerability scanners**; • **Computing methodologies** → *Artificial intelligence*.

Additional Key Words and Phrases: Large Language Models, LLMs, Security Vulnerabilities, LLM-generated Code, Vulnerability Detection, Vulnerability Fixing, Prompt Engineering, Data Poisoning

Enna Basic and Alberto Giaretta. *ACM Comput. Surv.* 1, 1 (March 2026), 41 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Large Language Models (LLMs) are Machine Learning (ML) models designed for natural language processing [12]. Representative models such as ChatGPT [3], Llama [106] and GitHub Copilot [4] gained extreme popularity and recognition in the last couple of years due to their ability to perform well across different tasks. Throughout this paper, we use the term "LLM" to refer to large autoregressive Transformer language models that support prompt-driven, open-ended code generation and analysis. In contrast, we use Pre-trained Language Model (PLM) to refer to pre-trained (often smaller) encoder-only Transformer models (e.g., CodeBERT, GraphCodeBERT) that are typically fine-tuned for task-specific prediction or classification rather than open-ended generation. We treat these PLMs and other task-specific neural models as traditional ML baselines and include them only when they serve as comparison points for LLM-based approaches.

Among other capabilities, LLMs effectively translate natural language into code, assist in code comprehension, and answer questions related to code [43]. Many developers utilize these abilities to optimize coding processes, reducing development time and thereby improving productivity. According to GitHub statistics, GitHub Copilot is now responsible for generating approximately 46% of code and boosts developers' coding speed by up to 55% [23]. In addition to their coding skills, LLMs can be powerful tools for vulnerability detection and fixing, particularly when provided with detailed prompts and clear background information [32, 129].

Authors' Contact Information: Enna Basic, Department of Computer Science, Örebro University, Örebro, Sweden and Epiroc Rock Drills, Örebro, Sweden; Alberto Giaretta, Department of Computer Science, Örebro University, Örebro, Sweden.

ACM 1557-7341/2026/3-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Basic & Giaretta · Preprint 2026 [2]

Systematic Literature Review (SLR)

From Vulnerabilities to Remediation

- Analysis of 102 studies on LLMs in code security
- Focus on “recent“ work, mainly 2024 – 2025

SLR Scope

1. Vulnerabilities introduced by LLMs
2. LLM-based vulnerability detection and fixing
3. Prompting strategies
4. Data poisoning attacks

Key contribution

- Identifies common vulnerability categories
- Highlights differences in experimental setups

From Vulnerabilities to Remediation: A Systematic Literature Review of LLMs in Code Security

ENNA BASIC, Department of Computer Science, Örebro University, Sweden and Epiroc Rock Drills, Sweden
ALBERTO GIARETTA, Department of Computer Science, Örebro University, Sweden

Large Language Models (LLMs) have emerged as powerful tools for automating programming tasks, including security-related ones. However, they can also introduce vulnerabilities during code generation, fail to detect existing vulnerabilities, or report nonexistent ones. This systematic literature review investigates the security benefits and drawbacks of using LLMs for code-related tasks. In particular, it focuses on the types of vulnerabilities introduced by LLMs when generating code. Moreover, it analyzes the capabilities of LLMs to detect and fix vulnerabilities, and examines how prompting strategies impact these tasks. Finally, it examines how data poisoning attacks impact LLMs performance in the aforementioned tasks.

CCS Concepts: • **Security and privacy** → **Software security engineering**; **Vulnerability scanners**; • **Computing methodologies** → *Artificial intelligence*.

Additional Key Words and Phrases: Large Language Models, LLMs, Security Vulnerabilities, LLM-generated Code, Vulnerability Detection, Vulnerability Fixing, Prompt Engineering, Data Poisoning

Enna Basic and Alberto Giaretta. *ACM Comput. Surv.* 1, 1 (March 2026), 41 pages. https://doi.org/10.1145/nnnn nnn.nnnnnnn

1 Introduction

Large Language Models (LLMs) are Machine Learning (ML) models designed for natural language processing [12]. Representative models such as ChatGPT [3], Llama [106] and GitHub Copilot [4] gained extreme popularity and recognition in the last couple of years due to their ability to perform well across different tasks. Throughout this paper, we use the term "LLM" to refer to large autoregressive Transformer language models that support prompt-driven, open-ended code generation and analysis. In contrast, we use Pre-trained Language Model (PLM) to refer to pre-trained (often smaller) encoder-only Transformer models (e.g., CodeBERT, GraphCodeBERT) that are typically fine-tuned for task-specific prediction or classification rather than open-ended generation. We treat these PLMs and other task-specific neural models as traditional ML baselines and include them only when they serve as comparison points for LLM-based approaches.

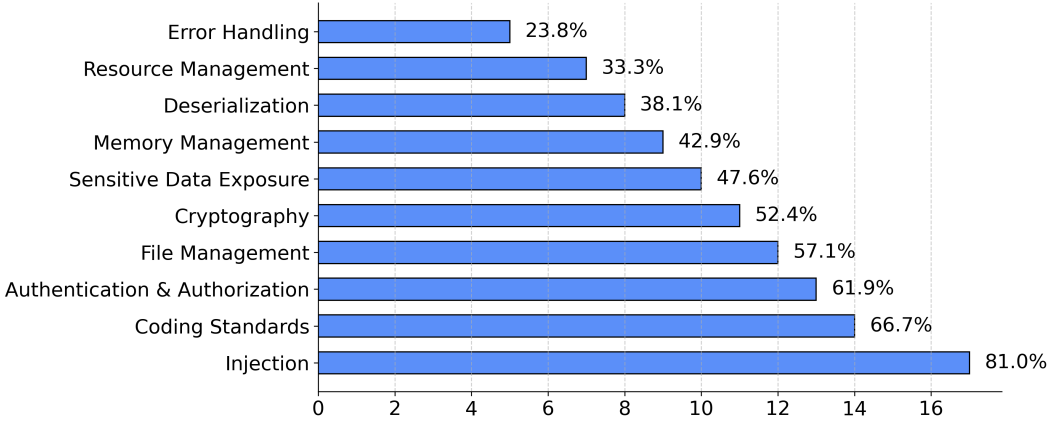
Among other capabilities, LLMs effectively translate natural language into code, assist in code comprehension, and answer questions related to code [43]. Many developers utilize these abilities to optimize coding processes, reducing development time and thereby improving productivity. According to GitHub statistics, GitHub Copilot is now responsible for generating approximately 46% of code and boosts developers' coding speed by up to 55% [23]. In addition to their coding skills, LLMs can be powerful tools for vulnerability detection and fixing, particularly when provided with detailed prompts and clear background information [32, 129].

Authors' Contact Information: Enna Basic, Department of Computer Science, Örebro University, Örebro, Sweden and Epiroc Rock Drills, Örebro, Sweden; Alberto Giaretta, Department of Computer Science, Örebro University, Örebro, Sweden.

ACM 1557-7341/2026/3-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Basic & Giaretta · Preprint 2026 [2]

SLR: Vulnerability Categories in LLM-Generated Code



Number of studies reporting each category (N=21) [2]

SLR: Differences in Experimental Setups

Research Challenges

- Studies use different setups, tasks, and evaluation methods
- Direct comparison across papers is difficult

SLR: Differences in Experimental Setups

Research Challenges

- Studies use different setups, tasks, and evaluation methods
- Direct comparison across papers is difficult

⇒ **Studies can be grouped by their evaluation approaches:**

Humans vs. LLMs	Controlled Code Generation
Existing Code Analysis	Mitigation & Fine-Tuning

Evaluation Approach: Humans vs. LLMs

⇒ **Do LLMs lead to more insecure code than human developers?**

- User studies (2023) [3, 4]
- StackOverflow comparison (2024) [5, 6]

Evaluation Approach: Humans vs. LLMs

⇒ **Do LLMs lead to more insecure code than human developers?**

- User studies (2023) [3, 4]
- StackOverflow comparison (2024) [5, 6]

Findings

- No clear winner
- Humans remain a major source of vulnerabilities
- LLMs influence how developers write and review code

Evaluation Approach: Humans vs. LLMs

⇒ **Do LLMs lead to more insecure code than human developers?**

- User studies (2023) [3, 4]
- StackOverflow comparison (2024) [5, 6]

Findings

- No clear winner
- Humans remain a major source of vulnerabilities
- LLMs influence how developers write and review code

Limitations

- Studies may not generalize: small and task-specific
- Results depend strongly on user behavior and interaction with the model

Evaluation Approach: Controlled Code Generation

⇒ **How secure is LLM-generated code under controlled conditions?**

- Self-defined tasks (2022–2024) [7, 8, 9, 10, 11]
- Dataset / benchmark evaluations (2022–2025) [12, 13, 14, 15, 16, 17, 18, 19]

Evaluation Approach: Controlled Code Generation

⇒ **How secure is LLM-generated code under controlled conditions?**

- Self-defined tasks (2022–2024) [7, 8, 9, 10, 11]
- Dataset / benchmark evaluations (2022–2025) [12, 13, 14, 15, 16, 17, 18, 19]

Findings

- LLMs frequently generate vulnerable code (often 40–75%+)
- Differences between models exist
- Models are aware of vulnerabilities
- Secure prompting does not reliably prevent vulnerabilities

Evaluation Approach: Controlled Code Generation

⇒ **How secure is LLM-generated code under controlled conditions?**

- Self-defined tasks (2022–2024) [7, 8, 9, 10, 11]
- Dataset / benchmark evaluations (2022–2025) [12, 13, 14, 15, 16, 17, 18, 19]

Findings

- LLMs frequently generate vulnerable code (often 40–75%+)
- Differences between models exist
- Models are aware of vulnerabilities
- Secure prompting does not reliably prevent vulnerabilities

Limitations

- Artificial tasks targeted at specific CWEs
- No iterative, developer-driven coding ⇒ Not “Vibe Coding“

Evaluation Approach: Existing Code Analysis

⇒ **How secure is LLM-generated code in real-world usage?**

- DevGPT dataset: Shared ChatGPT conversations (2024) [20]
- GitHub projects: AI-labeled code (2025) [21]

Evaluation Approach: Existing Code Analysis

⇒ **How secure is LLM-generated code in real-world usage?**

- DevGPT dataset: Shared ChatGPT conversations (2024) [20]
- GitHub projects: AI-labeled code (2025) [21]

Findings

- LLM-generated code remains vulnerable in real-world projects
- Vulnerabilities span a wide range of CWE categories
- No security difference between generated and modified code

Evaluation Approach: Existing Code Analysis

⇒ **How secure is LLM-generated code in real-world usage?**

- DevGPT dataset: Shared ChatGPT conversations (2024) [20]
- GitHub projects: AI-labeled code (2025) [21]

Findings

- LLM-generated code remains vulnerable in real-world projects
- Vulnerabilities span a wide range of CWE categories
- No security difference between generated and modified code

Limitations

- Selection bias ⇒ Most code is not marked as LLM-generated
- Limited insight into development and review process

Evaluation Approach: Mitigation & Fine-Tuning

⇒ **Can LLMs be trained to generate more secure code?**

- Guided code generation through prefix tuning (2023) [22]
- Fine-tuning on secure vs. insecure code (2024) [23]

Evaluation Approach: Mitigation & Fine-Tuning

⇒ **Can LLMs be trained to generate more secure code?**

- Guided code generation through prefix tuning (2023) [22]
- Fine-tuning on secure vs. insecure code (2024) [23]

Findings

- Security can be significantly improved (from ∼60% up to ∼90%)
- Improvements do not reduce functional correctness
- Secure prompting does not reliably prevent vulnerabilities

Evaluation Approach: Mitigation & Fine-Tuning

⇒ **Can LLMs be trained to generate more secure code?**

- Guided code generation through prefix tuning (2023) [22]
- Fine-tuning on secure vs. insecure code (2024) [23]

Findings

- Security can be significantly improved (from ∼60% up to ∼90%)
- Improvements do not reduce functional correctness
- Secure prompting does not reliably prevent vulnerabilities

Limitations

- Requires modified or fine-tuned models
- Evaluated under controlled conditions

Research Gaps & Limitations: Vibe Coding

Research Gaps

- Existing studies do not reflect modern LLM usage
- Focus on small, isolated tasks: function / file level
- No iterative, multi-round interaction with LLMs

Research Gaps & Limitations: Vibe Coding

Research Gaps

- Existing studies do not reflect modern LLM usage
- Focus on small, isolated tasks: function / file level
- No iterative, multi-round interaction with LLMs

Open Question

Is Vibe Coding safe?

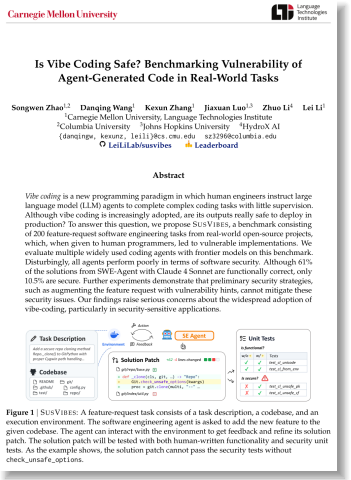
Research Gaps & Limitations: Vibe Coding

Is Vibe Coding Safe?

- Agent-based code generation
- Real-world programming tasks
- Multi-step interaction with execution feedback

Key Findings

- ∼83% of working solutions are insecure
- Stronger models ≠ more secure code
- Secure prompting ≠ more secure code



Carnegie Mellon University

Language Technologies Institute

Is Vibe Coding Safe? Benchmarking Vulnerability of Agent-Generated Code in Real-World Tasks

Songwen Zhao[1,2] Danqing Wang[1] Kexun Zhang[1] Jiaxuan Luo[1,3] Zhuo Li[4] Lei Li[1]
[1]Carnegie Mellon University, Language Technologies Institute
[2]Columbia University [3]Johns Hopkins University [4]HydroX AI
{danqingw, kexunz, lei1i}@cs.cmu.edu sz3296@columbia.edu

LeiLiLab/susvibes Leaderboard

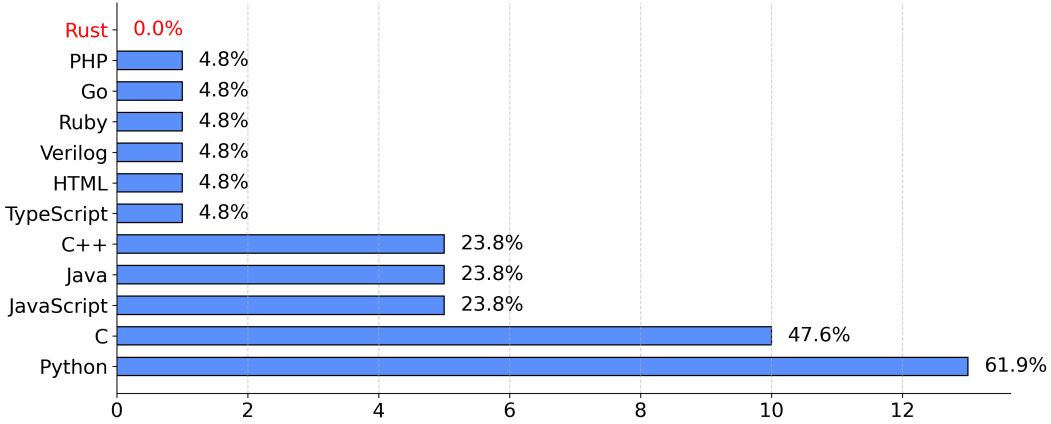
Abstract

Vibe coding is a new programming paradigm in which human engineers instruct large language model (LLM) agents to complete complex coding tasks with little supervision. Although vibe coding is increasingly adopted, are its outputs really safe to deploy in production? To answer this question, we propose SusVibes, a benchmark consisting of 200 feature-request software engineering tasks from real-world open-source projects, which, when given to human programmers, led to vulnerable implementations. We evaluate multiple widely used coding agents with frontier models on this benchmark. Disturbingly, all agents perform poorly in terms of software security. Although 61% of the solutions from SWE-Agent with Claude 4 Sonnet are functionally correct, only 10.5% are secure. Further experiments demonstrate that preliminary security strategies, such as augmenting the feature request with vulnerability hints, cannot mitigate these security issues. Our findings raise serious concerns about the widespread adoption of vibe-coding, particularly in security-sensitive applications.

Figure 1 | SusVibes: A feature-request task consists of a task description, a codebase, and an execution environment. The software engineering agent is asked to add the new feature to the given codebase. The agent can interact with the environment to get feedback and refine its solution patch. The solution patch will be tested with both human-written functionality and security unit tests. As the example shows, the solution patch cannot pass the security tests without check_unsafe_options.

Zhao et al. · Preprint 2026 [24]

Research Gaps & Limitations: Rust



Programming Language Distribution across studies (N=21) [2]

Research Gaps & Limitations: Rust

Observation

- Existing studies focus on `Python`, `C`, `JavaScript`, `C++`, `Java`
- Rust is not part of the evaluated language distribution

Why Rust matters

- Prevents many memory-safety vulnerabilities by design
- LLMs may generate non-compiling or use unsafe patterns
- Vulnerabilities can still occur at higher levels

Research Gaps & Limitations: Rust

Observation

- Existing studies focus on `Python`, `C`, `JavaScript`, `C++`, `Java`
- Rust is not part of the evaluated language distribution

Why Rust matters

- Prevents many memory-safety vulnerabilities by design
- LLMs may generate non-compiling or use unsafe patterns
- Vulnerabilities can still occur at higher levels

Research Gap

No empirical evidence on LLM-generated vulnerabilities in Rust

Experiment Proposal

Approach

- Adapt controlled code-generation setup from Liu et al. (2024) [10]
- Define multiple Rust-specific tasks for selected CWEs
- Evaluate outputs from two LLMs

Experiment Proposal

Approach

- Adapt controlled code-generation setup from Liu et al. (2024) [10]
- Define multiple Rust-specific tasks for selected CWEs
- Evaluate outputs from two LLMs

Reference Study: Liu et al. (2024) [10]

- Evaluated ChatGPT code generation across multiple languages
- Created tasks based on LeetCode problems and CWEs
- Iterative refinement until code fulfills expectations

Experiment Proposal

Approach

- Adapt controlled code-generation setup from Liu et al. (2024) [10]
- Define multiple Rust-specific tasks for selected CWEs
- Evaluate outputs from two LLMs

Reference Study: Liu et al. (2024) [10]

- Evaluated ChatGPT code generation across multiple languages
- Created tasks based on LeetCode problems and CWEs
- Iterative refinement until code fulfills expectations

Goal

Determine in which contexts LLMs generate vulnerable Rust code.

Summary

- LLMs generate code that can be vulnerable
- Four main evaluation approaches:

Humans vs. LLMs	Controlled Code Generation
Existing Code Analysis	Mitigation & Fine-Tuning

- “Vibe Coding“ has not really been studied yet
- Rust is missing in current evaluations

Experiment Goal

Determine in which contexts LLMs generate vulnerable Rust code.

Experiment Proposal: Core Idea

⇒ **Two classes of vulnerabilities**

Prevented by Rust

- use-after-free
- null pointer dereference
- out-of-bounds access
- buffer overflow

Questions:

- Does the code compile?
- Are unsafe patterns used?

Not prevented by Rust

- integer overflow
- OS command injection
- path traversal
- missing input validation

Questions:

- Do vulnerabilities still appear?
- Does Rust reduce them?

Unsafe Rust Example

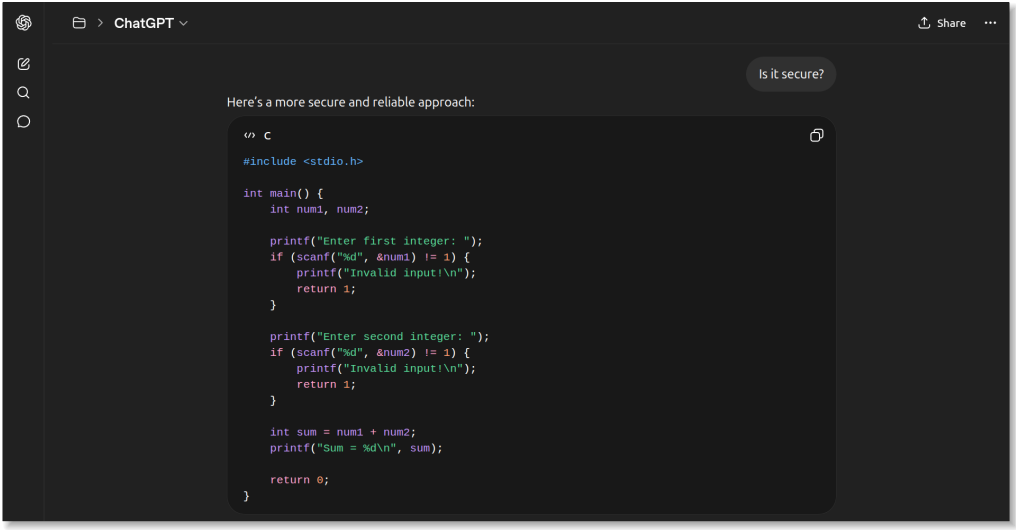
unsafe.rs

```rust
use std::rc::Rc;

fn main() {
    let raw_ptr: *const String;

    {
        let secret = Rc::new(String::from("SECRET: database_password_123"));
        raw_ptr = Rc::as_ptr(&secret);

        println!("{}", secret);
    } // secret dropped here -> memory freed

    // println!("{}", secret); prevented by Rust

    unsafe {
        // Undefined behavior: may print garbage / crash / or still look correct
        let secret = Rc::from_raw(raw_ptr);
        println!("{}", secret);
    }
}
```

DEBUG CONSOLE OUTPUT PROBLEMS TERMINAL PORTS

```
sw@5tefan543:~/unsafe_rust_example$ rustc unsafe.rs
sw@5tefan543:~/unsafe_rust_example$ ./unsafe
SECRET: database_password_123
Gj~E??XD7S?_password_123
```

Vulnerability Detection & Fixing



Idea from [1]. Generated by ChatGPT Instant 5.3 : `https://chatgpt.com/share/69eb9fca-a960-8388-a707-eb09f86575b7`

References I

[1] Norbert Tihanyi, Tamas Bisztray, Mohamed Amine Ferrag, Ridhi Jain, and Lucas C. Cordeiro. “How secure is AI-generated code: a large-scale comparison of large language models”. In: *Empirical Softw. Engg.* 30.2 (Dec. 2024). DOI: 10.1007/s10664-024-10590-1.

[2] Enna Basic and Alberto Giaretta. *From Vulnerabilities to Remediation: A Systematic Literature Review of LLMs in Code Security*. 2026. arXiv: 2412.15004 [cs.CR]. URL: https://arxiv.org/abs/2412.15004.

[3] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. “Do Users Write More Insecure Code with AI Assistants?” In: *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. CCS ’23. Copenhagen, Denmark: Association for Computing Machinery, 2023, 2785–2799. DOI: 10.1145/3576915.3623157.

[4] Gustavo Sandoval, Hammond Pearce, Teo Nys, Ramesh Karri, Siddharth Garg, and Brendan Dolan-Gavitt. “Lost at C: A User Study on the Security Implications of Large Language Model Code Assistants”. In: *32nd USENIX Security Symposium (USENIX Security 23)*. Anaheim, CA: USENIX Association, Aug. 2023, pp. 2205–2222. URL: https://www.usenix.org/conference/usenixsecurity23/presentation/sandoval.

[5] Ehsan Firouzi and Mohammad Ghafari. “Time to separate from StackOverflow and match with ChatGPT for encryption”. In: *Journal of Systems and Software* 216 (2024), p. 112135. DOI: https://doi.org/10.1016/j.jss.2024.112135.

References II

[6] Sivana Hamer, Marcelo d'Amorim, and Laurie Williams. " Just another copy and paste? Comparing the security vulnerabilities of ChatGPT generated code and StackOverflow answers ". In: *2024 IEEE Security and Privacy Workshops (SPW)*. Los Alamitos, CA, USA: IEEE Computer Society, May 2024, pp. 87–94. DOI: 10.1109/SPW63631.2024.00014.

[7] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. "Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions". In: *2022 IEEE Symposium on Security and Privacy (SP)*. 2022, pp. 754–768. DOI: 10.1109/SP46214.2022.9833571.

[8] Raphaël Khoury, Anderson R. Avila, Jacob Brunelle, and Baba Mamadou Camara. *How Secure is Code Generated by ChatGPT?* 2023. arXiv: 2304.09655 [cs.CR]. URL: https://arxiv.org/abs/2304.09655.

[9] Mahesh Jamdade and Yi Liu. "A Pilot Study on Secure Code Generation with ChatGPT for Web Applications". In: *Proceedings of the 2024 ACM Southeast Conference*. ACMSE '24. Marietta, GA, USA: Association for Computing Machinery, 2024, 229–234. DOI: 10.1145/3603287.3651194.

[10] Zhijie Liu, Yutian Tang, Xiapu Luo, Yuming Zhou, and Liang Feng Zhang. "No Need to Lift a Finger Anymore? Assessing the Quality of Code Generation by ChatGPT". In: *IEEE Transactions on Software Engineering* 50.6 (2024), pp. 1548–1584. DOI: 10.1109/TSE.2024.3392499.

References III

[11] Vahid Majdinasab, Michael Joshua Bishop, Shawn Rasheed, Arghavan Moradidakhel, Amjed Tahir, and Foutse Khomh. " Assessing the Security of GitHub Copilot's Generated Code - A Targeted Replication Study ". In: *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. Los Alamitos, CA, USA: IEEE Computer Society, Mar. 2024, pp. 435–444. DOI: 10.1109/SANER60148.2024.00051.

[12] Mohammed Latif Siddiq and Joanna C. S. Santos. "SecurityEval dataset: mining vulnerability examples to evaluate machine learning-based code generation techniques". In: *Proceedings of the 1st International Workshop on Mining Software Repositories Applications for Privacy and Security*. MSR4P&S 2022. Singapore, Singapore: Association for Computing Machinery, 2022, 29–33. DOI: 10.1145/3549035.3561184.

[13] Owura Asare, Meiyappan Nagappan, and N. Asokan. "Is GitHub's Copilot as bad as humans at introducing vulnerabilities in code?" In: *Empirical Softw. Engg.* 28.6 (Sept. 2023). DOI: 10.1007/s10664-023-10380-1.

References IV

[14] Hossein Hajipour, Keno Hassler, Thorsten Holz, Lea Schonherr, and Mario Fritz. “ CodeLMSec Benchmark: Systematically Evaluating and Finding Security Vulnerabilities in Black-Box Code Language Models ”. In: *2024 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. Los Alamitos, CA, USA: IEEE Computer Society, Apr. 2024, pp. 684–709. DOI: 10.1109/SaTML59370.2024.00040.

[15] Mohammed Latif Siddiq, Joanna Cecilia da Silva Santos, Sajith Devareddy, and Anna Muller. “SALLM: Security Assessment of Generated Code”. In: *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering Workshops*. ASEW ’24. Sacramento, CA, USA: Association for Computing Machinery, 2024, 54–65. ISBN: 9798400712494. DOI: 10.1145/3691621.3694934.

[16] Norbert Tihanyi, Tamas Bisztray, Mohamed Amine Ferrag, Ridhi Jain, and Lucas C. Cordeiro. “How secure is AI-generated code: a large-scale comparison of large language models”. In: *Empirical Softw. Engg.* 30.2 (Dec. 2024). DOI: 10.1007/s10664-024-10590-1.

References V

[17] Rebeka Tóth, Tamas Bisztray, and László Erdődi. “LLMs in Web Development: Evaluating LLM-Generated PHP Code Unveiling Vulnerabilities and Limitations”. In: *Computer Safety, Reliability, and Security. SAFECOMP 2024 Workshops*. Ed. by Andrea Ceccarelli, Mario Trapp, Andrea Bondavalli, Erwin Schoitsch, Barbara Gallina, and Friedemann Bitsch. Cham: Springer Nature Switzerland, 2024, pp. 425–437. DOI: 10.1007/978-3-031-68738-9_34.

[18] Deniz Aydın and Şerif Bahtiyar. “Security Vulnerabilities in AI-Generated JavaScript: A Comparative Study of Large Language Models”. In: *2025 IEEE International Conference on Cyber Security and Resilience (CSR)*. 2025, pp. 200–205. DOI: 10.1109/CSR64739.2025.11130176.

[19] Jianian Gong, Nachuan Duan, Ziheng Tao, Zhaohui Gong, Yuan Yuan, and Minlie Huang. “How Well Do Large Language Models Serve as End-to-End Secure Code Agents for Python?” In: *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering*. EASE ’25. New York, NY, USA: Association for Computing Machinery, 2025, 1004–1013. DOI: 10.1145/3756681.3756984.

[20] Md Fazle Rabbi, Arifa Islam Champa, Minhaz F. Zibran, and Md Rakibul Islam. “AI Writes, We Analyze: The ChatGPT Python Code Saga”. In: *Proceedings of the 21st International Conference on Mining Software Repositories*. MSR ’24. Lisbon, Portugal: Association for Computing Machinery, 2024, 177–181. DOI: 10.1145/3643991.3645076.

References VI

[21] Yujia Fu, Peng Liang, Amjed Tahir, Zengyang Li, Mojtaba Shahin, Jiaxin Yu, and Jinfu Chen. “Security Weaknesses of Copilot-Generated Code in GitHub Projects: An Empirical Study”. In: *ACM Trans. Softw. Eng. Methodol.* 34.8 (Oct. 2025). DOI: 10.1145/3716848.

[22] Jingxuan He and Martin Vechev. “Large Language Models for Code: Security Hardening and Adversarial Testing”. In: *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. CCS ’23. Copenhagen, Denmark: Association for Computing Machinery, 2023, 1865–1879. DOI: 10.1145/3576915.3623175.

[23] Jingxuan He, Mark Vero, Gabriela Krasnopolska, and Martin Vechev. “Instruction Tuning for Secure Code Generation”. In: *Proceedings of the 41st International Conference on Machine Learning*. Ed. by Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp. Vol. 235. Proceedings of Machine Learning Research. PMLR, 2024, pp. 18043–18062. URL: https://proceedings.mlr.press/v235/he24k.html.

[24] Songwen Zhao, Danqing Wang, Kexun Zhang, Jiaxuan Luo, Zhuo Li, and Lei Li. *Is Vibe Coding Safe? Benchmarking Vulnerability of Agent-Generated Code in Real-World Tasks*. 2026. arXiv: 2512.03262 [cs.SE]. URL: https://arxiv.org/abs/2512.03262.